

PROJECT NEXELIS

INTERN-TO-ENGINEER BOOTCAMP

DAY 8 — POSTGRESQL & DATABASE FUNDAMENTALS

Prepared By: Shivam Jha

Role: Software Engineering Intern

1. Objective:-

The objective of Day 8 is to understand how **application data is permanently stored, organized, related, and managed using PostgreSQL.**

The practical work focuses on designing and implementing an **Employee Management System** using relational database concepts such as:

- Tables, rows, and columns
- Primary Keys and Foreign Keys
- Constraints and data integrity
- Entity relationships and cardinality
- Normalization
- Indexing
- Transactions

Key Objective:-

Design a structured and reliable PostgreSQL database that can store Employee Management System data while maintaining relationships and data integrity.

The objective of Day 8 was to understand persistent data storage using PostgreSQL. I applied relational database concepts by designing separate tables for Employee, Department, Designation, and Manager and connecting them through appropriate keys and constraints

2. Practical Task

The practical task for Day 8 was to **design and implement a relational database for an Employee Management System using PostgreSQL.**

Entities:-

The database consists of four main entities:

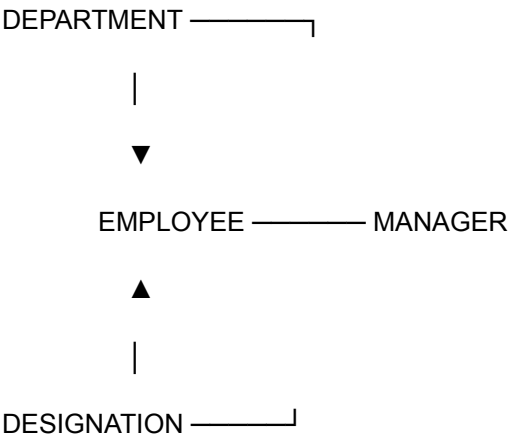
Entity	Purpose
Employee	Stores employee information
Department	Stores department information
Designation	Stores job designation information
Manager	Stores manager information linked to employees

Implementation Requirements:-

The database was designed with:

- Primary Keys for unique record identification
- Foreign Keys for table relationships
- **NOT NULL** constraints for required data
- **UNIQUE** constraints to prevent duplicate values
- Appropriate PostgreSQL data types
- Sample data for testing
- ER diagram representing the database relationships

Database Structure



For the practical task, I designed an Employee Management System database in PostgreSQL with four entities: Employee, Department, Designation, and Manager. I implemented the tables with primary keys, foreign keys, constraints, and sample data, and represented their relationships through an ER diagram.

3. Tools & Technologies

Tool / Technology	Purpose
PostgreSQL	Relational database management and persistent data storage
pgAdmin 4	Creating tables, executing SQL, and inspecting database records
SQL	Database definition, data insertion, and data retrieval
VS Code	Writing and managing SQL scripts
Mermaid	Creating the Entity Relationship Diagram (ERD)
Git & GitHub	Version control and project repository management

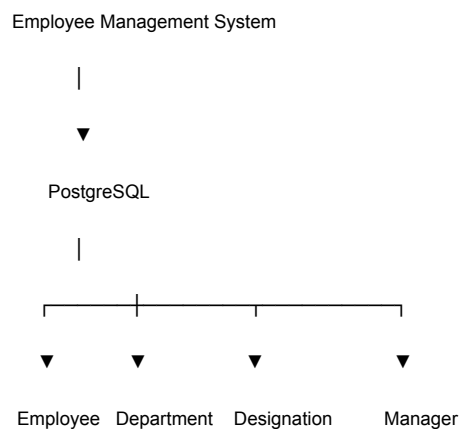
4. Database

4.1 Database:-

A **database** is an organized collection of structured data that enables an application to **store, retrieve, update, and manage information persistently**.

In a relational database, data is stored in **tables**, and related tables are connected using **keys and relationships**.

Employee Management System



4.2 Purpose

The database provides the **persistent storage layer** of the application.

Without Database:-

Application → Memory → Application Stops → Data Lost

With Database:-

Application → PostgreSQL → Persistent Storage → Data Available

4.3 Key Characteristics

A database should provide:

- **Persistence** — data remains stored after the application stops.
- **Integrity** — invalid data is prevented.
- **Consistency** — data remains accurate and reliable.
- **Relationships** — related data can be connected.

- **Efficient Querying** — required data can be retrieved efficiently.
- **Concurrency** — multiple users can access data safely.
- **Transactions** — related operations can be handled reliably.
- **Scalability** — the system can handle increasing data and users.

4.4 Database Operations

Database operations are commonly represented by **CRUD**:

Operation	SQL Command	Purpose
Create	INSERT	Add new data
Read	SELECT	Retrieve data
Update	UPDATE	Modify existing data
Delete	DELETE	Remove data

Example:-

-- Create

```
INSERT INTO department (department_name)
```

```
VALUES ('IT');
```

-- Read

```
SELECT * FROM department;
```

-- Update

```
UPDATE department
```

```
SET department_name = 'Information Technology'
```

```
WHERE department_id = 1;
```

```
-- Delete
```

```
DELETE FROM department
```

```
WHERE department_id = 1;
```

4.5 Database Integrity Example

In the Employee table:

```
department_id INT NOT NULL
```

This ensures that an employee **cannot be stored without a department**, allowing the database to enforce important rules at the data layer.

A database is the persistent storage layer of an application. In my Employee Management System, PostgreSQL stores employee, department, designation, and manager data in structured relational tables. The database also maintains data integrity through constraints and relationships and supports CRUD operations for managing the data.

5. PostgreSQL

5.1 PostgreSQL:-

PostgreSQL is an open-source relational database management system (RDBMS) used to store, organize, retrieve, and manage structured data.

It uses SQL (Structured Query Language) to create tables, manage records, define relationships, and perform database operations.

5.2 Purpose

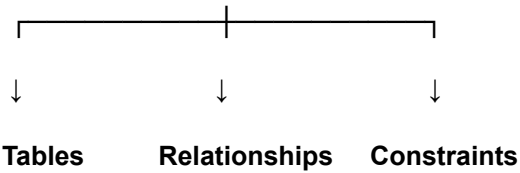
PostgreSQL provides the database environment where the Employee Management System stores its data.

Employee Management System

↓

PostgreSQL

↓



5.3 Why PostgreSQL?

PostgreSQL provides:

- Relational data storage
- Primary & Foreign Keys
- Constraints
- Transactions
- SQL querying
- Data integrity
- Concurrent access
- Scalability

5.4 PostgreSQL in the Project

The Employee Management System uses PostgreSQL to manage four main tables:

DEPARTMENT

DESIGNATION

EMPLOYEE

MANAGER

6. Schema

6.1 Schema:-

A **schema** is a logical structure inside a database that organizes and contains database objects such as **tables, relationships, views, and other objects**.

In PostgreSQL, a database can contain multiple schemas.

Database

|

└─ Schema

|

└─ Table

└─ Table

└─ Table

6.2 Purpose

A schema helps to:

- **Organize** database objects
- **Separate** different application or business areas
- **Manage permissions** at an organizational level
- **Avoid naming conflicts**
- Keep larger databases structured

6.3 PostgreSQL Example

PostgreSQL commonly provides a default schema called **public**.

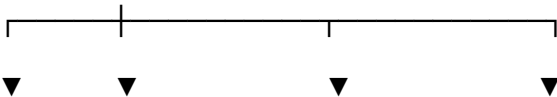
Your project tables are created inside this schema unless another schema is specified.

Employee Management Database

|

public

|



Employee Department Designation Manager

For example:

```
CREATE TABLE public.employee (  
    employee_id SERIAL PRIMARY KEY,  
    employee_name VARCHAR(100) NOT NULL
```


);

Here, `public.employee` means the **employee table inside the public schema**.

6.4 Database vs Schema

Database	Schema
Contains the overall database environment	Organizes objects inside a database
Can contain multiple schemas	Can contain multiple tables
Higher-level container	Logical namespace

A schema is a logical container used to organize database objects within a PostgreSQL database. In my project, the tables are created under the default `public` schema, which keeps the Employee Management System database objects organized.

7. Table

7.1 Table:-

A **table** is a database object used to store related data in a structured format of **rows and columns**.

In a relational database, each table generally represents a particular **entity or business concept**.

Example

In the Employee Management System:

EMPLOYEE TABLE

```
employee_id | employee_name | email | salary
-----|-----|-----|-----
```

1	Rahul Sharma	rahul@company.com	60000
2	Amit Kumar	amit@company.com	85000

Here:

- **Table** → `employee`
- **Columns** → `employee_id`, `employee_name`, `email`, `salary`
- **Rows** → Individual employee records

7.2 Purpose

Tables help to:

- Organize related data
- Store records systematically
- Apply constraints to data
- Establish relationships with other tables
- Retrieve and manage data using SQL

7.3 Tables in Your Project

Your Employee Management System contains four main tables:

Table	Stores
<code>department</code>	Department information
<code>designation</code>	Job designation information
<code>employee</code>	Employee information
<code>manager</code>	Manager information

Each table has its own structure and responsibility.

7.4 Creating a Table

Example from project:

```
CREATE TABLE department (  
    department_id SERIAL PRIMARY KEY,  
    department_name VARCHAR(100) NOT NULL UNIQUE  
);
```

This defines the table structure and specifies rules for its columns.

A table is a structured database object that stores related data in rows and columns. In my Employee Management System, I separated the data into Department, Designation, Employee, and Manager tables, with each table having a specific responsibility.

8. Row

8.1 ROW:-

A row is a single record in a database table. It contains the complete set of values for one entity.

Example

In the **employee** table:

employee_id	employee_name	email	salary
1	Rahul Sharma	rahul@company.com	60000

The highlighted line represents one row, or one employee record.

8.2 Purpose

Rows are used to:

- Store individual records
- Represent a specific entity
- Retrieve specific information

- Update existing records
- Delete records

For example:

```
SELECT * FROM employee
```

```
WHERE employee_id = 1;
```

This retrieves the row belonging to employee ID 1.

8.3 Row Operations

INSERT → Add a new row

SELECT → Read rows

UPDATE → Modify a row

DELETE → Remove a row

8.4 Row in Your Project

If your **employee** table contains 25 employees, it contains 25 employee rows.

Each row represents one employee and contains values such as:

- Employee ID
- Name
- Email
- Phone
- Salary
- Joining Date
- Department ID
- Designation ID

A row represents a single record in a database table. In my Employee Management System, each row in the Employee table represents one employee and contains all the corresponding values for that employee.

9. Column

9.1 COLUMN:-

A column represents a specific attribute or field of the data stored in a table.

Each column has a name and a data type that defines what kind of value it can store.

Example

In the **employee** table:

employee_id | employee_name | email | salary

-----|-----|-----|-----

1 | Rahul Sharma | rahul@company.com | 60000

Here:

- **employee_id** → identifies the employee
- **employee_name** → stores employee name
- **email** → stores email address
- **salary** → stores salary

These are columns.

9.2 Purpose

Columns help to:

- Define the structure of a table
- Store a specific type of information
- Apply appropriate data types
- Apply constraints and validation
- Make data easier to query and manage

9.3 Column with Data Type

Example from your project:

employee_name VARCHAR(100) NOT NULL

salary NUMERIC(10,2)

joining_date DATE

Here:

- **employee_name** → column name
- **VARCHAR(100)** → data type

- **NOT NULL** → constraint

A column represents a specific attribute of an entity in a database table. Each column has a defined data type and can have constraints. In my Employee table, columns such as employee_name, email, salary, and joining_date store specific information about each employee.

10. Primary Key (PK)

10.1 PK:-

A Primary Key (PK) is a column, or combination of columns, that uniquely identifies each record in a table.

A Primary Key must be:

- Unique — duplicate values are not allowed.
- NOT NULL — every record must have a value.

Example

employee_id SERIAL PRIMARY KEY

Here, **employee_id** uniquely identifies each employee.

10.2 Purpose

Primary Keys are used to:

- Uniquely identify records
- Prevent duplicate identities
- Maintain entity integrity
- Provide a reference point for Foreign Keys
- Identify the correct record during **UPDATE** and **DELETE**

10.3 Primary Keys in Project

Table	Primary Key
-------	-------------

department	department_id
------------	---------------

designatio
n designation_i
 d

employee employee_id

manager manager_id

Example

```
CREATE TABLE employee (  
    employee_id SERIAL PRIMARY KEY,  
    employee_name VARCHAR(100) NOT NULL  
);
```

SERIAL automatically generates sequential integer IDs for new records.

10.4 PK and Foreign Key Relationship

A Primary Key can be referenced by a Foreign Key in another table.

DEPARTMENT

department_id (PK)

↑

|

| referenced by

|

EMPLOYEE

department_id (FK)

This establishes a relationship between the two tables.

A Primary Key uniquely identifies each record in a table and does not allow duplicate or NULL values. In my Employee Management System, I used primary keys for all four tables. These keys uniquely identify records and also provide the basis for establishing relationships through Foreign Keys.

11. Foreign Key (FK)

11.1 FK:-

A Foreign Key (FK) is a column that creates a relationship between two tables by referencing the Primary Key of another table.

In simple terms: Foreign Key connects related tables.

11.2 Purpose

Foreign Keys are used to:

- Establish relationships between tables
- Maintain referential integrity
- Prevent references to non-existing records
- Connect related business data

11.3 Example

In our database:

department_id INT NOT NULL,

FOREIGN KEY (department_id)

REFERENCES department(department_id)

Here:

DEPARTMENT

department_id (PK)

↑

|

| Foreign Key

EMPLOYEE

department_id (FK)

The Employee table refers to a valid Department.

11.4 Foreign Keys in Your Project

Table	Foreign Key	References
employee	department_id	department.department_id
employee	designation_id	designation.designation_id
manager	employee_id	employee.employee_id

This connects your four database entities.

11.5 Referential Integrity

Suppose Department ID 1 exists:

department_id = 1 → IT

An employee can reference:

department_id = 1

But an employee cannot reference a department that does not exist, such as:

department_id = 99

This protects the database from invalid relationships.

A Foreign Key is used to establish a relationship between two tables by referencing the Primary Key of another table. In my Employee Management System, I use Foreign Keys to connect Employee with Department and Designation, and Manager with Employee. This maintains referential integrity and prevents invalid references.

12. Constraints

12.1 Constraints

Constraints are rules applied to database columns to control the type and validity of data stored in a table.

They help the database prevent invalid or inconsistent data.

12.2 Common Constraints

Constraint	Purpose
PRIMARY KEY	Uniquely identifies each record
FOREIGN KEY	Maintains relationships between tables
NOT NULL	Prevents empty/NULL values
UNIQUE	Prevents duplicate values
CHECK	Ensures values satisfy a condition
DEFAULT	Provides a value when none is supplied

12.3 Constraints in Project

Example from the **department** table:

```
CREATE TABLE department (
```

```
department_id SERIAL PRIMARY KEY,  
  
department_name VARCHAR(100) NOT NULL UNIQUE  
  
);
```

Here, three rules are applied:

- **PRIMARY KEY** → **department_id** must be unique.
- **NOT NULL** → department name is required.
- **UNIQUE** → duplicate department names are not allowed.

For the Employee table:

```
department_id INT NOT NULL
```

This ensures every employee must have a department.

12.4 Why Constraints Matter

Constraints provide database-level data integrity.

Instead of depending only on application validation:

Application



Validation



PostgreSQL



Constraints



Valid Data

Even if incorrect data reaches PostgreSQL, the database can reject it when it violates a constraint.

Constraints are database-level rules used to maintain data integrity. In my Employee Management System, I used Primary Key, Foreign Key, NOT NULL, and UNIQUE constraints to prevent invalid, missing, duplicate, or incorrectly related data.

13. Relationships

13.1 Relationships:-

A relationship defines how data in one table is connected to data in another table.

In a relational database, relationships are usually established using Primary Keys and Foreign Keys.

13.2 Why Relationships Are Needed

Relationships help to:

- Connect related data
- Avoid unnecessary duplication
- Maintain data consistency
- Retrieve information across multiple tables
- Represent real-world business relationships

13.3 Common Types of Relationships

1. One-to-One (1:1)

One record in Table A is related to one record in Table B.

Employee ——— Manager

1 1

2. One-to-Many (1:N)

One record can be related to many records.

Example:

Department

|

|— Employee

|— Employee

|— Employee

One department can have many employees.

3. Many-to-Many (M:N)

Many records in one table can relate to many records in another table.

Example:

A doctor can treat many patients:

Doctor A — Patient 1

— Patient 2

— Patient 3

At the same time, a patient can be treated by multiple doctors:

Patient 1 — Doctor A

— Doctor B

Therefore:

Many Doctors ↔ Many Patients

13.4 Relationships in Project

Your Employee Management System primarily uses:

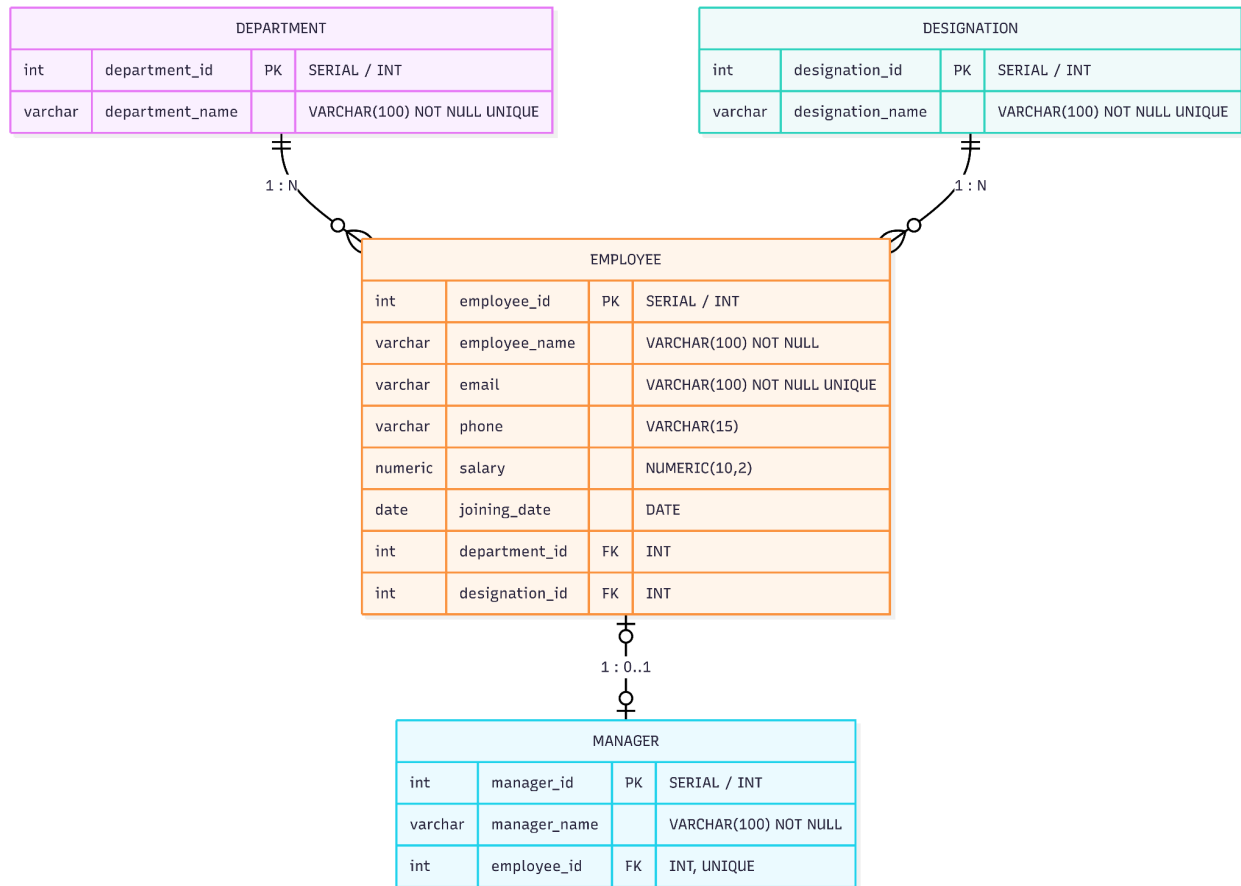
Department —→ Employee

Designation —→ Employee

Employee —→ Manager

The `department_id` and `designation_id` Foreign Keys in `employee` establish these connections.

A relationship defines how entities in different tables are connected. In my Employee Management System, Department and Designation are connected to Employee through Foreign Keys, while Manager is linked to Employee. These relationships help organize related data without unnecessary duplication.



14. UNIQUE Constraint

14.1 UNIQUE Constraint

The **UNIQUE** constraint ensures that no two records contain the same value in a particular column.

It is used when a value must be unique but is not the primary identifier of the record.

Example — User Email

email VARCHAR(100) **UNIQUE**

This allows:

rahul@gmail.com

amit@gmail.com

neha@gmail.com

But prevents:

rahul@gmail.com

rahul@gmail.com ❌

14.2 UNIQUE in Project

I used **UNIQUE** for department names:

department_name VARCHAR(100) NOT NULL UNIQUE

Therefore, the database will not allow two departments with the same name.

I also used it for:

email VARCHAR(100) NOT NULL UNIQUE

So two employees cannot have the same email address.

14.3 PRIMARY KEY vs UNIQUE

Primary Key	UNIQUE
Identifies the record	Ensures value is unique
Cannot be NULL	Can allow NULL depending on database design
Normally one PK constraint per table	Multiple UNIQUE constraints can exist
Used as the main identity	Used for other unique attributes

The **UNIQUE** constraint prevents duplicate values in a column. In my Employee Management System, I used it for employee email and department name because these values should not be duplicated. Unlike a Primary Key, **UNIQUE** is used to enforce uniqueness for specific business attributes.

15. NULL & NOT NULL

15.1 NULL & NOT NULL

NULL represents the absence of a value or that the value is currently unknown/not provided.

NOT NULL is a constraint that requires a column to always contain a value.

NULL does not mean 0 or an empty string ('').

15.2 Example

phone VARCHAR(15)

The phone number can be NULL if it is not provided.

But:

employee_name VARCHAR(100) NOT NULL

means an employee must have a name.

These represent different situations.

15.3 NULL in Project

My employee table contains:

employee_name VARCHAR(100) NOT NULL,

email VARCHAR(100) NOT NULL UNIQUE,

phone VARCHAR(15),

salary NUMERIC(10,2),

joining_date DATE,

department_id INT NOT NULL,

designation_id INT NOT NULL

Therefore:

- Employee name must be provided

- Email must be provided
- Department must be provided
- Designation must be provided
- Phone, salary, and joining date can be NULL

This allows the database design to distinguish between required and optional information.

NULL represents the absence of a value, while NOT NULL ensures that a value must be provided. In my Employee Management System, I used NOT NULL for essential fields such as employee name, email, department, and designation, while optional fields such as phone can remain NULL.

16. Indexing

16.1 Indexing

An index is a database data structure that helps PostgreSQL find and retrieve records faster.

It works like the index of a book. Instead of checking every page to find a topic, you use the index to directly locate where the information is.

Index = Faster data searching and retrieval.

16.2 Why Do We Need an Index?

Suppose the **employee** table has 1 million records and we run:

```
SELECT *  
  
FROM employee  
  
WHERE email = 'rahul@company.com';
```

Without a suitable index

PostgreSQL may need to examine many rows to find the matching employee.

Query

↓

Search many rows

↓

Find matching record

With an index

PostgreSQL can use the index to locate the relevant record more efficiently.

Query



Index



Find matching location



Retrieve record

So indexes are particularly valuable when tables become large and columns are frequently searched or joined.

16.3 Creating an Index

For example:

```
CREATE INDEX idx_employee_email  
ON employee(email);
```

This creates an index on the `email` column.

Now PostgreSQL has an additional structure that can help with queries involving that column.

16.4 Important Difference

Primary Key $\neq$ Index

A Primary Key is a data integrity and identification rule.

An Index is a performance structure used to make certain queries faster.

PostgreSQL creates an index to support a Primary Key, but the concepts are different.

An index is a database data structure used to improve the performance of data retrieval. It works similar to the index of a book, allowing the database to locate relevant records more efficiently instead of scanning the entire table. In my project, PostgreSQL automatically creates indexes for Primary Key and UNIQUE constraints. Additional indexes should be created based on frequently executed queries because indexes improve reads but add storage and write-maintenance overhead.

17. Normalization

17.1 Normalization

Normalization is a database design technique used to organize data into logical and related tables. It helps reduce data redundancy, prevent data anomalies, and maintain data consistency.

Instead of repeatedly storing the same information, related information is separated and connected using keys.

Example — Unnormalized Design

Order ID	Customer	Product	Supplier
101	Arjun	Laptop	Dell
102	Arjun	Laptop	Dell
103	Priya	Monitor	LG

Here, customer, product, and supplier information may be repeated unnecessarily.

Normalized Design

CUSTOMER

customer_id	customer_name
1	Arjun
2	Priya

PRODUCT

product_id	product_name	supplier_id
101	Laptop	1
102	Monitor	2

ORDER

order_id	customer_id	product_id
1001	1	101
1002	1	101
1003	2	102

The information is stored in appropriate tables and connected using IDs.

17.2 Objectives of Normalization

Normalization helps to:

- Reduce duplicate data
- Maintain data consistency
- Prevent insertion, update, and deletion anomalies
- Simplify database maintenance
- Create clear table relationships
- Improve database organization

17.3 First Normal Form (1NF)

Rule

A table is in 1NF when:

- Each field contains a single, atomic value.
- There are no repeating groups.

Example

Not in 1NF:

Employee

Skills

In 1NF:

Employee	Skill
Arjun	Python
Arjun	SQL
Arjun	AWS

Key Point:

1NF = One value per field

17.4 Second Normal Form (2NF)

Rule

A table must:

1. Be in 1NF
2. Have every non-key attribute fully dependent on the complete primary key

This is mainly relevant when a table has a composite primary key.

Example

Consider:

Student ID	Course ID	Student Name	Course Name	Marks
101	C01	Arjun	DBMS	85

Primary Key:

(Student_ID, Course_ID)

Here:

- **Student_Name** depends only on **Student_ID**
- **Course_Name** depends only on **Course_ID**
- **Marks** depends on both

This creates **partial dependency**.

Solution

Separate the data into:

STUDENT

Student ID	Student Name
101	Arjun

COURSE

Course ID	Course Name
C01	DBMS

RESULT

Student ID	Course ID	Marks
101	C01	85

Key Point:

2NF = No partial dependency

17.5 Third Normal Form (3NF)

Rule

A table must:

1. Be in 2NF

2. Have no dependency of a non-key attribute on another non-key attribute.

Example

Not in 3NF:

Employee ID	Department ID	Department Name
101	10	Engineering
102	10	Engineering

Dependency:

Employee_ID $\rightarrow$ Department_ID

Department_ID $\rightarrow$ Department_Name

Therefore, **Department_Name** depends indirectly on the employee.

Solution

EMPLOYEE

Employee ID	Department ID
101	10
102	10

DEPARTMENT

Department ID	Department Name
---------------	-----------------

10	Engineering
----	-------------

Key Point:

3NF = No transitive dependency

17.6 Normal Forms — Summary

Normal Form	Main Purpose
1NF	Atomic values
2NF	Remove partial dependency
3NF	Remove transitive dependency
BCNF	Determinants must be superkeys
4NF	Remove independent multi-valued dependencies
5NF	Remove unnecessary join dependencies

17.7 Normalization in Employee Management System

In the Employee Management System, information is separated according to its business entity:

DEPARTMENT

|

▼

EMPLOYEE

▲

|

DESIGNATION

EMPLOYEE

|

▼

MANAGER

For example, department information is maintained separately rather than repeatedly storing the department name for every employee.

The Employee table references the department using:

```
department_id INT NOT NULL
```

This results in a cleaner and more maintainable relational design.

Normalization is a database design technique used to reduce redundancy and prevent data anomalies. It organizes data into logically related tables and progresses through normal forms from 1NF to 5NF, with each level addressing specific dependency and redundancy problems. In my Employee Management System, I separated entities such as Department, Designation, Employee, and Manager and connected them using keys, resulting in a more structured and consistent database design.

18. Transactions

18.1 Transactions

A transaction is a sequence of one or more database operations that are treated as a single logical unit of work.

A transaction ensures that related operations are completed safely. If the transaction succeeds, changes are saved; if an error occurs, changes can be undone.

Transaction = All related operations succeed together or the changes are rolled back.

18.2 Why Transactions Are Required

Transactions are important when multiple operations are dependent on each other.

Example — Bank Transfer

A ₹5,000 transfer involves two operations:

Account A

↓

Deduct ₹5,000

↓

Add ₹5,000

↓

Account B

If the deduction succeeds but adding the amount fails, the database must not remain in a partially updated state.

A transaction handles this safely:

BEGIN



Deduct ₹5,000



Add ₹5,000



COMMIT

If something goes wrong:

BEGIN



Operation fails



ROLLBACK

18.3 Transaction Control Commands

Command	Purpose
BEGIN	Starts a transaction
COMMIT	Permanently saves the changes
ROLLBACK	Cancels uncommitted changes
SAVEPOINT	Creates a point to which a transaction can partially roll back

Example

BEGIN;

UPDATE employee

SET salary = 70000

WHERE employee_id = 1;

UPDATE employee

SET salary = 75000

WHERE employee_id = 2;

COMMIT;

If an error occurs before **COMMIT**:

ROLLBACK;

18.4 ACID Properties

Transactions are commonly explained using the ACID properties:

Property	Meaning	Simple Explanation
Atomicity	All or nothing	Either all required operations happen or none do
Consistency	Valid state	Database rules remain satisfied

Isolation	Independent execution	Concurrent transactions should not improperly affect each other
Durability	Permanent result	Committed changes survive failures

18.5 Transaction in Project

Our Employee Management System mainly performs CRUD operations.

Transactions become especially useful when multiple related changes must be completed together.

For example:

Employee Operation



Multiple Database Changes



Transaction



Success Failure



COMMIT ROLLBACK

This prevents the database from being left in a partially updated state.

18.6 Key Benefits

- Maintains data consistency
- Prevents partial updates

- Supports safe multi-step operations
- Provides recovery through rollback
- Protects data integrity during concurrent operations

A transaction is a group of related database operations treated as one logical unit of work. It ensures that the database does not remain in a partially updated state. PostgreSQL provides transaction control using commands such as **BEGIN**, **COMMIT**, **ROLLBACK**, and **SAVEPOINT**, while the ACID properties ensure reliability and consistency.